第 13 讲：二分查找——利用单调性每次排除一半

0. 本讲的核心问题

在排好序的数组中查找一个目标，线性扫描需要最坏 `O(n)` 时间。

如果查看中间元素：

- 中间值太小，左半部分全部不可能；
- 中间值太大，右半部分全部不可能。

每次把搜索范围缩小约一半，这就是二分查找。

二分查找的本质不是“数组中间取一个数”，而是：

> 搜索空间具有单调性，因此一次判断能够安全排除一整半候选。

除了有序数组，二分还可以用于“答案范围”：判断某个答案是否可行，并利用可行性的单调变化寻找边界。

---

1. 为什么时间是 `O(log n)`

搜索范围长度变化：

```
n → n/2 → n/4 → n/8 → ... → 1
```

经过 `k` 次后：

```
n / 2^k ≈ 1
```

因此：

```
k ≈ log₂(n)
```

即使数组有一百万个元素，二分也只需要大约 20 次判断。

---

2. 最重要的选择：区间定义

二分最常见错误不是算法思想，而是区间边界不一致。

常用两套写法：

1. 闭区间 `[left, right]`；
2. 左闭右开 `[left, right)`。

两套都正确，但循环条件、更新方式必须与区间定义匹配。

本讲主要使用闭区间：

```
left     right
```

---

3. 标准二分：查找一个等于 target 的位置

```python
def binary_search(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        middle = left + (right - left) // 2

        if nums[middle] == target:
            return middle
        elif nums[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1
```

3.1 循环不变量

每轮开始时：

> 如果 target 存在，那么它一定在闭区间 `[left, right]` 中。

3.2 为什么循环条件是 `left <= right`

闭区间在 `left == right` 时仍包含一个候选元素，必须检查。

当 `left > right` 时，闭区间为空，说明目标不存在。

3.3 为什么更新要加减 1

`middle` 已经检查过且不等于目标，下一轮必须排除它：

```python
left = middle + 1
right = middle - 1
```

如果写成 `left = middle`，当区间只剩两个元素时可能永远不缩小，导致死循环。

---

4. 用表格手动跟踪

数组：

```
[1, 3, 5, 7, 9]
```

查找 `7`：

left	right	middle	nums[middle]	操作
0	4	2	5	太小，`left = 3`
3	4	3	7	找到

遇到二分边界问题时，不要凭感觉修改代码。选择：

- 空数组；
- 单元素；

- 两元素；
- 目标在首尾；
- 目标不存在；

逐轮写出 `left / middle / right`。

---

5. 左闭右开写法

了解另一套写法有助于阅读代码：

```
[python]
def binary_search_half_open(nums, target):
    left = 0
    right = len(nums)

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] == target:
            return middle
        elif nums[middle] < target:
            left = middle + 1
        else:
            right = middle

    return -1
```

因为右端不包含在搜索范围中，所以排除中点及右侧时可以写：

```
[python]
right = middle
```

学习阶段建议选定一套主模板，不要在同一道题里混用。

---

6. 查找边界：第一个大于等于 target 的位置

数组可能有重复值：

```
[1, 2, 2, 2, 5]
```

普通二分找到任意一个 `2` 就返回，但有时需要第一个 `2`。

更通用的定义是：

> 找到第一个满足 `nums[i] >= target` 的位置。

这叫 lower bound。

```
[python]
def lower_bound(nums, target):
    left = 0
    right = len(nums)

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] < target:
            left = middle + 1
        else:
            right = middle
```

```
    return left
```

这里使用左闭右开区间 `[left, right)`，并允许答案等于 `len(nums)`，表示所有元素都小于 target。

6.1 为什么遇到等于时不能返回

当前 `middle` 虽然满足条件，但左侧可能还有更早的满足位置。应保留 `middle`，继续搜索左半部分：

```
[python]
right = middle
```

6.2 搜索插入位置

`lower_bound` 的结果正是把 target 插入有序数组且保持有序的位置：

```
[python]
def search_insert(nums, target):
    return lower_bound(nums, target)
```

---

7. 查找第一个大于 target 的位置

定义：

> 找到第一个满足 `nums[i] > target` 的位置。

```
[python]
def upper_bound(nums, target):
    left = 0
    right = len(nums)

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] <= target:
            left = middle + 1
        else:
            right = middle

    return left
```

重复值区间可以由：

```
[python]
first = lower_bound(nums, target)
after_last = upper_bound(nums, target)
```

得到。

如果 `first == after_last`，说明 target 不存在。

最后一个等于 target 的下标是：

```
[python]
after_last - 1
```

前提是先确认 target 存在。

---

8. 边界二分的统一理解：False 到 True

把每个位置是否满足条件写成布尔值。

例如查找第一个 `>= target`：

```
nums:       [1, 2, 2, 4, 7]
>= target:   F  F  F  T  T
```

问题变成：

> 找到单调布尔序列中第一个 True。

通用模板：

```
[python]
def first_true(left, right, condition):
    #    [left, right)        right
    while left < right:
        middle = left + (right - left) // 2

        if condition(middle):
            right = middle
        else:
            left = middle + 1

    return left
```

这个模板是“答案二分”的基础。

---

9. 旋转排序数组

原有序数组：

```
[0, 1, 2, 4, 5, 6, 7]
```

旋转后：

```
[4, 5, 6, 7, 0, 1, 2]
```

整体不再有序，但每次取中点后，至少有一半仍然有序。

```
[python]
def search_rotated(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        middle = left + (right - left) // 2

        if nums[middle] == target:
            return middle

        if nums[left] <= nums[middle]:
            #
            if nums[left] <= target < nums[middle]:
                right = middle - 1
            else:
                left = middle + 1
        else:
            #
            if nums[middle] < target <= nums[right]:
                left = middle + 1
            else:
                right = middle - 1

    return -1
```

思考顺序：

```
1.
```

```
2. target
3.        target
```

这个版本通常假设元素互不重复。大量重复值会使“哪一半有序”的判断失去区分度，需要额外处理。

---

10. 寻找旋转数组最小值

无重复元素时，可以与右端值比较：

```
[python]
def find_min_rotated(nums):
    left = 0
    right = len(nums) - 1

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] > nums[right]:
            left = middle + 1
        else:
            right = middle

    return nums[left]
```

如果 `nums[middle] > nums[right]`，最小值一定在 middle 右侧；否则 middle 可能就是最小值，所以保留它并令 `right = middle`。

注意这里的目标是边界，不是寻找某个等于值，因此循环和更新与标准二分不同。

---

11. 二维矩阵二分

若矩阵每行有序，并且下一行首元素大于上一行末元素，可以把它看成一个长度为 `rows * cols` 的虚拟一维数组。

一维下标 `middle` 映射到：

```
[python]
row = middle // cols
col = middle % cols
```

```
[python]
def search_matrix(matrix, target):
    if not matrix or not matrix[0]:
        return False

    rows = len(matrix)
    cols = len(matrix[0])
    left = 0
    right = rows * cols - 1

    while left <= right:
        middle = left + (right - left) // 2
        row = middle // cols
        col = middle % cols
        value = matrix[row][col]

        if value == target:
            return True
        if value < target:
            left = middle + 1
        else:
            right = middle - 1

    return False
```

不需要真的创建扁平数组。

---

12. 在答案范围上二分

有些题没有给出可直接搜索的有序数组，但答案存在一个数值范围，并且“某个答案是否可行”具有单调性。

例如：在限定天数内运送包裹，求最小运载能力。

若能力为 `capacity`：

- 太小可能无法按时完成；
- 某能力一旦可行，更大的能力也都可行。

形成：

```
F  F  F  T  T  T
```

目标是第一个 True。

12.1 解题步骤

```
1.
2.    can_finish(answer)
3.
4.
```

---

13. 示例：在 D 天内运送包裹

每天按顺序装载，不能改变包裹顺序。求最低运载能力。

答案边界：

```
[python]
left = max(weights)      #
right = sum(weights)     #
```

判断某个能力需要多少天：

```
[python]
def days_needed(weights, capacity):
    days = 1
    current_load = 0

    for weight in weights:
        if current_load + weight > capacity:
            days += 1
            current_load = 0

        current_load += weight

    return days
```

二分最小可行能力：

```
[python]
def ship_within_days(weights, days):
    left = max(weights)
    right = sum(weights)

    while left < right:
        middle = left + (right - left) // 2

        if days_needed(weights, middle) <= days:
```

```
                right = middle
            else:
                left = middle + 1

        return left
```

13.1 为什么可行时保留 middle

`middle` 已经可行，但它可能不是最小可行值，因此令：

```
[python]
right = middle
```

继续向左寻找边界。

总复杂度：

```
 O(n)
 O(log(answer_range))
O(n log(answer_range))
```

---

14. 怎样识别答案二分

看到“最大化最小值”或“最小化最大值”时可以考虑，但不能只靠关键词。

真正需要确认：

1. 答案可以在一个有序范围中枚举；
2. 给定候选答案，可以较快判断是否可行；
3. 可行性是单调的。

常见问题：

- 最小运载能力；
- 最低吃香蕉速度；
- 分配任务的最小最大负载；
- 能放置物体的最大最小距离。

如果可行性不是单调的，不能直接二分答案。

---

15. 常见错误

15.1 混用区间定义

先在纸上写：

```
[left, right]   [left, right)
```

再决定循环条件和更新。

15.2 搜索范围不缩小

闭区间标准查找必须排除 middle：

```
[python]
left = middle + 1
right = middle - 1
```

15.3 忽略单元素区间

闭区间使用 `while left <= right`。写成 `<` 会漏掉最后一个候选。

15.4 找边界时遇到等于就返回

若要求第一个或最后一个位置，等于只说明当前是候选，还要继续向边界搜索。

15.5 答案范围上下界错误

答案二分要先证明边界：左端可能不可行，右端通常应保证可行；或者使用明确的 first-true 区间约定。

15.6 `can_finish` 不具有单调性

先写出一串 False/True 示意，再证明只会发生一次转变。

15.7 空输入直接访问首尾

在读取 `nums[0]`、`matrix[0]` 前处理题目约束或空输入。

---

16. 本讲检查表

```
1.
2.
3.
4. left right
5.
6. middle
7.               /
8. condition(middle)
9.
10.
```

17. 本讲总结

1. 二分依赖单调性，每次判断排除约一半候选。
2. 区间定义必须与循环条件、边界更新保持一致。
3. 标准闭区间查找使用 `left <= right` 和 `middle ± 1`。
4. 边界问题可统一为寻找单调布尔序列中的第一个 True。
5. 旋转数组每轮至少有一半有序，可以据此排除。
6. 答案二分需要“范围 + 判断函数 + 单调性证明”。